like rendering or analytics pipelines. The trade-off is that each codebase must properly integrate Cedar and maintain consistent request generation and policy evaluation logic. Standardized integration patterns and libraries, shared policy stores, and caching support can mitigate this trade-off somewhat.

MTLS-ONLY AND NETWORK CONTROLS

In some cases, coarse-grained controls are enough. Services can be limited to communicate only with specific peers using workload identity from mTLS certificates or network rules, like Kubernetes NetworkPolicy. For example, you might allow “Docs can call Search” but not “Search can call HR.” This method is efficient and simple, but it can’t enforce fine-grained rules such as “Docs cannot index Legal documents.” Many organizations set up these controls as a baseline and then layer fine-grained authorization on top for resource-level decisions.

EVENT OR BROKER ENFORCEMENT

In asynchronous architectures, a message broker like Kafka, NATS, and SQS serves as the natural enforcement point. Brokers can enforce coarse-grained rights such as “Service A may publish to Topic X” or “Service B may consume from Queue Y.” For fine-grained checks—for example, whether a specific message complies with sensitivity requirements—the authorization engine typically operates on the consumer side. One thing to watch out for: While brokers can prevent unauthorized producers or consumers, sensitive data may still propagate unless consumers apply additional policy checks per message.

DIRECT TOKEN EXCHANGE

Instead of relying on proxies or a mesh, a calling service can obtain a short-lived token from a security token service (STS) and present it directly to the callee. The callee verifies the token and consults an authorization engine before proceeding. This approach is flexible in smaller systems, but each service must handle identity and authorization logic consistently. This pattern is typically used for coarse-grained controls.